5. Project Development Phase

FitBuddy – AI Fitness Plan Generator using Gemini Models

Phase Report

Prepared from the supplied FitBuddy project documentation and organized into a project lifecycle deliverable.

1. Development Environment

Use Python and a virtual environment. Install the packages listed in requirements.txt. Store secrets in a local .env file and keep that file out of source control.

2. Suggested Project Structure

FitBuddy-AI/ app/ (main.py, routes.py, config.py, database.py, models.py, schemas.py, gemini_generator.py, gemini_flash_generator.py, updated_plan.py); templates/ (index.html, result.html, all_users.html); static/css/style.css; requirements.txt; .env.example; README.md.

3. Backend Development

Create the FastAPI application, configure Jinja2 templates and static files, define the home route, plan-generation route, feedback route, and admin route. Use FastAPI Form parameters for HTML form data and validate inputs before calling AI services.

4. Gemini Integration

Create a shared configuration layer that reads the API key from environment variables. Use the selected Gemini model for structured workout generation and plan updates, and a fast model for nutrition/recovery tips. Ask the model to return clear, readable, day-by-day output.

5. Database Development

Use SQLAlchemy models and SQLite. Implement helpers to create or retrieve users, save original plans, store feedback and updated plans, and query all records for the admin page.

6. Frontend Development

Build index.html with labeled input fields and goal/intensity selections. Build result.html to show user details, workout plan, nutrition tip, and feedback form. Build all_users.html to list user details and original/updated plans. Add responsive CSS.

7. Configuration

Provide .env.example with a placeholder such as GOOGLE_API_KEY=your_gemini_api_key_here. Never commit a real API key. Keep runtime configuration centralized in app/config.py.

8. Development Verification

Run the server with: uvicorn app.main:app --reload. Open http://127.0.0.1:8000 for the web interface and http://127.0.0.1:8000/docs for API documentation.

Note: This document describes the intended design and workflow. Test outcomes should be recorded only after the application has been run and verified.